

LEARNING & STUDY REFERENCE

The AI Stack

Ten layers of a modern AI system, explained — and mapped to Prompt AI News, Katie, and Hermes.

Prepared by Carlos Lucero • promptainews.com
Reference date: August 2026

This is a study reference, not a product spec. Each layer below is explained the way it would be taught in an AI-systems course, then grounded in the actual pipelines I run day to day — the Prompt AI News publishing stack and my two personal chatbots, Katie (local, private) and Hermes Agent (local, agentic). The goal is to understand *why* each layer exists, not just what tool fills it, so the knowledge survives the next tool swap.

At a Glance: The Ten Layers

Layer	What it does	Where it shows up for me
1. Compute	Runs the actual inference	RTX 5060 (local) + Anthropic cloud (Claude)
2. Data	Sources and stores raw material	RSS feeds, dedup JSON logs, Sanity CMS
3. Foundation Models	The models doing the reasoning	Claude Sonnet (cloud), hermes3/dolphin3 (local)
4. Orchestration / Agents	Chains steps into a workflow	News → rewrite → post → review pipeline
5. Memory & Context	Carries state across runs	seen_stories.json, ai_learning_state.json

Layer	What it does	Where it shows up for me
6. Tools & Integrations	Lets a model act in the world	Sanity API, Gmail SMTP, TextBelt, MCP
7. Application / Interface	Where a human meets the system	promptainews.com, Sanity Studio, WhatsApp
8. Observability	Shows what's actually happening	Grafana/Prometheus/Jaeger, analytics digest
9. Governance & Oversight	Where a person makes the call	Manual Publish click, writing-quality rules
10. Security & Secrets	Protects credentials and access	Windows user-scope env vars

Layers marked **critical now & into the future** in the sections that follow are the highest-leverage places to keep learning — not because the others don't matter, but because these are moving fastest and compound the most.

01 Compute / Infrastructure Layer

WHAT IT IS

The physical or rented hardware that actually executes model inference and code — CPUs, GPUs, RAM, and increasingly, cloud capacity billed by the second.

HOW IT SHOWS UP IN MY SYSTEMS

My desktop (Intel i5-14400F, RTX 5060, 8GB VRAM) runs Ollama locally for Katie and Hermes Agent. That 8GB VRAM ceiling is a hard wall: it caps me to 8B-parameter Q4 models with roughly a 16K context window — pushing past that (I tried 131K) causes a CUDA out-of-memory crash. For anything bigger or higher-stakes, I lean on Anthropic's cloud compute: Claude Sonnet runs the Prompt AI News Daily Draft Publisher routine and this Claude Code session.

WHY IT MATTERS

Compute is the layer with the hardest physical ceiling of the ten — you can't out-clever a VRAM limit. Local compute buys privacy and zero marginal cost per run; cloud compute buys capability that won't fit on one GPU. The durable skill is knowing which task belongs on which side of that line, not owning more hardware.

02 Data Layer

WHAT IT IS

The raw material — how information is sourced, captured, and stored (and de-duplicated) before anything intelligent happens to it.

HOW IT SHOWS UP IN MY SYSTEMS

Twelve-plus RSS feeds pour into three sourcing agents (`ai_news_agent.py`, `china_ai_agent.py`, `ev_news_agent.py`). Separate `seen_*.json` files act as 24-hour dedup ledgers so the same story never gets sent twice, and `promptainews.com` is explicitly filtered out of its own sourcing so the site never reports on itself. Sanity CMS is the structured store for everything published; Umami captures visitor data.

WHY IT MATTERS

“Garbage in, garbage out” still governs LLMs completely — a more capable model does not fix a noisy feed. The unglamorous plumbing (dedup, self-exclusion, source filtering) is what keeps three automated digests a day from turning into spam.

03 Foundation Model Layer

WHAT IT IS

The language models actually doing the reasoning and generation — the component most people mean when they say “AI.”

HOW IT SHOWS UP IN MY SYSTEMS

Claude Sonnet (cloud) rewrites raw news into journalist-quality drafts and runs this Claude Code session. Locally, Ollama hosts `hermes3`, `llama3.1:8b`, `dolphin3`, and `qwen2.5:7b` (`qwen3:14b` is installed but too large for this GPU). `hermes3` powers Hermes Agent's cron jobs; `dolphin3:8b`, fine-tuned via a custom `Modelfile`, is Katie.

WHY IT MATTERS

This is the layer everyone associates with “AI,” but it's also the most commoditized and the most likely to be swapped under you — my nyt-digest cron job broke silently when a provider switch changed which model it was pinned to. Increasingly, durable value sits in the layers built around the model, not the model itself.

04 Orchestration / Agent Layer

WHAT IT IS

The logic that chains model calls into a multi-step workflow — deciding what happens first, second, third, and what triggers each step.

HOW IT SHOWS UP IN MY SYSTEMS

This is where Prompt AI News actually lives: Layer 1 sourcing feeds Layer 2 (a cloud routine that reads the digest, rewrites it, and posts drafts to Sanity via a Vercel proxy), which feeds Layer 3 (analytics/alerts) and Layer 4 (my manual review and publish). Hermes Agent's cron subsystem runs the identical pattern locally for personal jobs like the morning briefing.

CRITICAL NOW & INTO THE FUTURE

This is the fastest-moving layer in the entire stack. A year ago, “AI” meant one prompt and one response. Now it means agents that read email, decide what's newsworthy, call APIs, and text me — with no human touch until the review gate I've deliberately kept in place. Getting good at orchestration, not just prompting, is the single highest-leverage skill for where this is all heading.

05 Memory & Context Layer

WHAT IT IS

How a system remembers anything beyond a single conversation turn — state, history, and retrieved context fed back into future runs.

HOW IT SHOWS UP IN MY SYSTEMS

seen_stories.json is short-term memory with a 24-hour expiry. ai_learning_state.json tracks a rotating 60-topic curriculum so Hermes's morning briefing never repeats a lesson. Claude's own memory system — the one behind this document — is the long-horizon version of the same idea: durable facts that persist across sessions instead of resetting with every script run.

RISING IN IMPORTANCE

Context windows are still finite (my local 16K ceiling is a hard wall, not a preference), so what a system remembers versus discards is a design decision, not an afterthought. As agents run longer and more autonomously, memory becomes the difference between a system that improves over time and one that starts from zero every single run.

06 Tools & Integrations Layer (APIs)

WHAT IT IS

The connective tissue — the APIs and protocols that let a model or agent act outside its own head: send an email, post to a CMS, hit an SMS gateway.

HOW IT SHOWS UP IN MY SYSTEMS

The Sanity API and the `/api/create-drafts` Vercel proxy, Gmail SMTP, TextBelt SMS, and Bing IndexNow all sit in this layer, alongside the MCP connections (Gmail, Calendar, Drive, Notion) wired into the cloud routine. Hermes uses the same pattern to reach WhatsApp and Telegram.

WHY IT MATTERS

MCP (Model Context Protocol) is standardizing this layer industry-wide: instead of every integration being bespoke glue code, tools become swappable, discoverable connectors. That's the direction this is heading — fewer one-off scripts, more standard plumbing.

07 Application / Interface Layer

WHAT IT IS

Where a human actually meets the system — the website, the chat window, the dashboard.

HOW IT SHOWS UP IN MY SYSTEMS

promptainews.com (Next.js, Vercel, Tailwind) is the public-facing product. Sanity Studio is my editorial interface for review and publish. WhatsApp and Telegram are the interface for Katie and Hermes.

WHY IT MATTERS

This is the layer readers and users actually judge you on — everything below it is invisible if this layer is polished, and nothing below it matters if this layer is broken.

08 Observability & Monitoring Layer

WHAT IT IS

Instrumentation that shows what a system is actually doing — traces, metrics, logs, and alerts — as opposed to what you assume it's doing.

HOW IT SHOWS UP IN MY SYSTEMS

The local chatbot stack runs OpenTelemetry → Jaeger (traces) and Prometheus (metrics) → Grafana (dashboards), plus a Telegram-based monitor.py that pages me on health issues. Prompt AI News has its own version: Umami analytics, a daily analytics digest email, and plain task logs (`ai_brief.log`, `analytics.log`).

RISING IN IMPORTANCE

The more autonomous a system gets, the more it needs eyes on it that aren't yours. This is the layer that catches a silently-broken cron job — like the nyt-digest provider-pin issue — before it fails quietly for a week without anyone noticing.

09 Governance & Human Oversight Layer

WHAT IT IS

The deliberate checkpoints where a person, not a model, makes the final call.

HOW IT SHOWS UP IN MY SYSTEMS

Every Prompt AI News story is an unpublished draft until I personally review it and click Publish in Sanity Studio — no automation publishes directly to the site. Writing-quality standards (mandatory source attribution, a banned-phrase list, no AI slop) are written down and enforced as a standard, not a vibe.

CRITICAL NOW & INTO THE FUTURE

This layer decides whether “autonomous” is a feature or a liability. The roadmap item to eventually allow a fully auto-publish pipeline is explicitly gated on trust being earned first — that's the correct order of operations, and it's the layer most people skip when they get excited about automation.

10 Security & Secrets Layer

WHAT IT IS

How credentials, tokens, and access are stored and scoped so a compromised script can't compromise everything.

HOW IT SHOWS UP IN MY SYSTEMS

Umami and TextBelt credentials were moved out of hardcoded plaintext and into Windows user-scope environment variables, read via `os.environ` in both `analytics_digest.py` and `country_watch.py`. One open item remains on purpose: a Gmail app password is still hardcoded in `analytics_digest.py`, a known and deliberately deferred tradeoff rather than an oversight.

WHY IT MATTERS

As more of this stack runs unattended, every hardcoded secret is a larger blast radius if a script or repo ever leaks. It's the least visible layer and the easiest to neglect — which is exactly why it deserves deliberate attention instead of being handled ad hoc, one script at a time.

+ Where To Focus Next

Three layers are worth deliberate study over the next year, in this order:

- **Orchestration / Agents** — This is what turns a chatbot into the Prompt AI News pipeline. The planned “custom writing skill” and eventual auto-publish mode both live here.
- **Governance & Oversight** — The harder half of automation isn't building the pipeline — it's deciding when to trust it. Every expansion of autonomy should come with an equal expansion of review.
- **Memory & Context** — Longer-running agents are only as good as what they remember between runs. This is the layer under the most active development industry-wide right now.

Everything in this reference is grounded in the System Bible for Prompt AI News (Google Drive, updated August 1, 2026) and the local chatbot setups for Katie and Hermes Agent. Update this document as the stack changes — it's meant to be re-printed, not filed away.